

```
pip install adafruit-ampy
pip install adafruit-ampy --upgrade
```

Note. In some cases, you might face issue with picocom. In that case, you can use any serial terminal and monitor to install the pico framework. Also, after flashing the MicroPython, the board may sometimes not get connected to Wi-Fi and the internet and show an error. In that case you need to follow the instructions given under the network setup section below.

Setting up the network and connecting to Wi-Fi

Open the serial terminal and give the commands mentioned below to connect to the Wi-Fi network:

```
import network
sta_if = network.WLAN(network.STA_IF)
ap_if = network.WLAN(network.AP_IF)
sta_if.active(True)
sta_if.connect('<your SSID>', '<your
key>') # Replace the SSID with your
Wi-Fi network name and key with your
network password
sta_if.isconnected()
sta_if.ifconfig()
```

Now, with the Node MCU connected to Wi-Fi, we can check its IP address using following command and note the IP address:

```
sta_if.ifconfig()

It gives an output like:
('192.168.0.2', '255.255.255.0',
'192.168.0.1', '8.8.8.8')
```

Setting PicoWeb framework of MicroPython

We are now ready to install the PicoWeb, which is a micro web framework for MicroPython. If you are acquainted with Flask web framework in Python, you will find it similar to work with. Two important features of PicoWeb are:

1. Asynchronous, which means it can handle multiple concurrent requests.
2. Small memory usage.

To install the framework in Node MCU, clone the repo to a location in your system using the code:

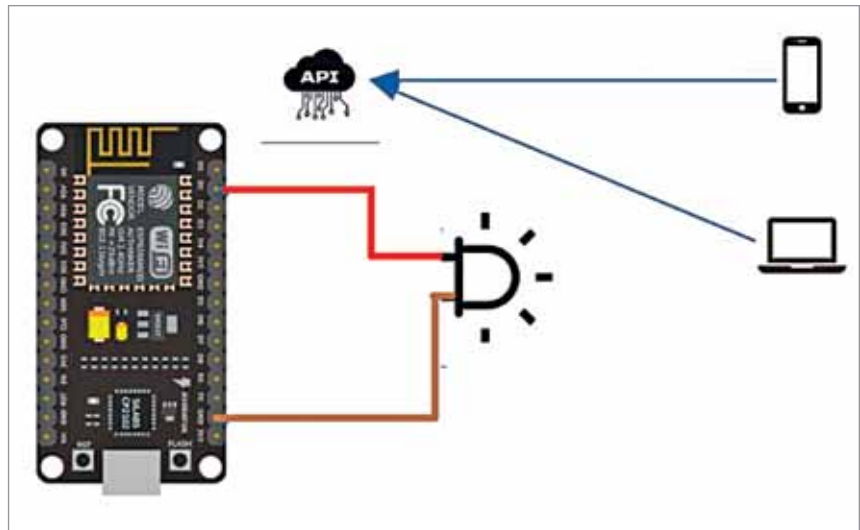


Fig. 1: Connection and working

```
$ git clone https://github.com/
peterhinch/MicroPython-samples
```

Dependencies of PicoWeb framework

To install the framework onto the board, we need to first install a few dependencies. Open the REPL prompt, as mentioned earlier, and install the packages as mentioned below:

```
$ import upip
$ upip.install('MicroPython-uasyncio')
$ upip.install('MicroPython-ulogging')
$ upip.install('MicroPython-pkg_
resources')
$ upip.install('utemplate')
```

Step out of REPL, using `ctl + A + X`, and copy some file from the local system into the board. Move to the folder where the PicoWeb repo from git is downloaded, check out for the files `__init__.py` and `utils.py`. Copy these files to the PicoWeb directory on the board. To do this, make use of the other cli tool, `adafruit-ampy`, which was installed earlier.

Step 1. Create the PicoWeb directory using the code:

```
$ ampy --port /dev/ttyUSB0 mkdir picoweb
```

Step 2. Copy the files from the local system to the PicoWeb directory on the board using the code:

```
$ ampy --port /dev/ttyUSB0 put __init.py
```

```
/picoweb/__init__.py
$ ampy --port /dev/ttyUSB0 put utils.py
/picoweb/utils.py
```

Creating API endpoints using PicoWeb framework

Now, we need to create the Python script by the name `main.py`, which will be done in two steps. First, by connecting to the available Wi-Fi and, second, by creating the api endpoint to control GPIO pin state of the board. We will also create an access point (AP) for the board, so that if there is no Wi-Fi available, we can still connect to the board AP and control the state of the GPIO pins.

```
sudo nano main.py
```

Now, copy the following code in `main.py` file:

```
import picoweb
import network
from machine import Pin
import time

ip=[]
app = picoweb.WebApp(__name__)

#create the Access Point
ap = network.WLAN(network.AP_IF)
ap.active(True)
ap.config(essid='espAP',
password='espAPpassword')
myip = ap.ifconfig()
```